

Sewa Setu

Agentic Application Scrutiny & Document Verification

- An agentic pipeline that pre-scrutinizes every certificate application and hands the officer a ranked, explained report - while the decision stays with the officer.

Team: <fill in names, roles, emails>

Repository: <https://github.com/sharifmdathar/sewa-setu>

Live demo: <deployment link, if applicable>

Demo credentials: none - the prototype has no auth by design (SPEC §3)

The problem is scrutiny time, not application time

A certificate application takes an officer 12-18 minutes to scrutinize by hand.

- Two officers reading the same file can reach two different decisions.
- Forged, expired and mismatched documents get through under volume pressure.
- Every application is re-read from scratch: identity, dates, amounts, duplicates.

The bottleneck is not the citizen submitting - it is the official reading.

What we built

Extraction -> rule checks -> adjudication -> fraud scoring -> recommendation

- A VLM or template parser reads each uploaded document into typed fields.
- Five deterministic checks (C1-C5) run over the application and its documents.
- Only the checks a rule cannot settle go to the LLM adjudicator, which re-words the verdict for an officer - it never overturns evidence.
- A fraud scorer weights the outcomes into riskScore 0-100 and one recommendation.

The agent recommends. The officer decides. That boundary is enforced in code and in tests.

Five checks, and no verdict without evidence

- C1 identity match - the same person across every document and the declared fields
- C2 document validity - expiry, issue order, plausible issuing authority
- C3 completeness - every required document type present and legible
- C4 cross-field consistency - declared amounts against stated amounts
- C5 fraud signals - duplicate file hashes across applicants, order-of-ten amounts

Real C5 evidence from the demo store:

- "APP-0014-fee_receipt-5.txt has hash 8c7ce8117b88 which also appears in another application; APP-0014-self_income_declaration-4.txt states 4240000, exactly an order of ten away from the declared 424000." -> riskScore 100, recommend reject

RESULTS

Evaluation on 200 synthetic applications

Metric	Manual baseline	This run	Target
Fail-flag precision	unmeasured	1.000	≥ 0.90
Fail-flag recall	unmeasured	1.000	≥ 0.85
Risk-flag recall	unmeasured	0.540	disclosed, not gated
Officer wait per application	12-18 min	3.0 ms p50	< 60 s

- 917 documents, 50 applications with planted anomalies, seed 42.

74 planted check failures: 74 caught, 0 false alarms, 0 missed. The gate asks whether each defect lit up exactly the check that owns it.

- Risk-flag recall is 0.540 and we disclose it: single-anomaly applications score 40-50, under the spec-pinned threshold of 60. The queue ranks; it does not catch.

We put a real model in the loop, and reported what it did

Same corpus, real vision model (llama-3.2-11b-vision), 20 rendered documents:

- 107 of 124 stated field values read correctly - recall 0.86, precision 0.98
- 5 of 20 documents read with no omission at all
- Control, same documents through the deterministic parser: 124 of 124

The failure is not uniform. Identity fields read at 1.00; expiryDate at 0.22.

- We built a targeted second-pass fix for it, measured it recovering nothing, and reverted it rather than shipping a call that buys nothing.

What shipped instead costs zero calls: C2 warns when an expiry-carrying document yields no expiry, so a null becomes an officer-visible gap instead of a silent pass.

Latency: three different true answers

What was timed	Result
Pipeline in-process, offline, no HTTP	0.12 ms per application
The officer's real wait over HTTP (rules-only)	p50 3.0 ms p95 3.2 ms (100 runs)
One document through a live VLM	p50 2.8 s p95 18.2 s max 23.8 s
One application through a live VLM	13.3 s mean 15.5 s worst (2 apps)

Which number is honest depends on what was timed. We quote the officer's wait.

Why the numbers can be trusted

310 automated tests passing, 2 skipped pending a paid model key.

- Contract conformance is judged by a validator against openapi.yaml itself - in-process and over a real HTTP socket, not against a hand-written expectation.
- Every eval figure regenerates from one command, and each report titles itself rules-only or live model leg so the two can never be quoted for each other.
- A local OpenAI-compatible endpoint in the test suite asserts request counts over a socket - that is how we found a retry budget being multiplied, and fixed it.

4,080 lines pipeline · 1,402 eval harness · 3,179 web · 1,052 generator · 5,794 test lines

Contract-first, two tracks, one week

FastAPI pipeline + Next.js officer and citizen UI, joined by one frozen OpenAPI contract of 10 paths.

- Two people worked in separate paths without editing each other's files.
- Five change requests raised; three applied in a paired integration session.
- Nothing personal is logged, and the corpus is fictional by construction - names, IDs and addresses come from generated pools.

The contract held everywhere it spoke. Where it was silent - applicantFields is a bare object - both tracks guessed differently and both test suites passed. We found it, fixed it, and wrote CR-5 to close the hole.

Pilot plan

P0 - what this is

- Two services, synthetic evidence, officer shadow mode. The pipeline runs beside the manual process; officers see the report and the queue but decide alone. Nothing touches a registry, so the worst case is a wrong recommendation an officer overrides.

P1 - next

- Choose an OCR/VLM vendor on measured accuracy, not on a demo. Registry adapters behind the same extractor interface - which one runs is configuration, not a call site.

P2 - at scale

- Multilingual, mobile, a real database. The store here is single-process by design.

What we are asking for

An anonymised real document set, so the eval can run against scans rather than renders - our 0.86 read-back figure is a ceiling, not an expectation.

- A vendor endpoint to measure accuracy and cost on that set.
- Officer time to validate the recommendation format, because the ranking is the thing they will argue with - and a 0.54 risk recall gives them reason to.

Repository: <https://github.com/sharifmdathar/sewa-setu>

Live demo: <deployment link> Video: <demo video link>

- Synthetic data only. No real PII, no registry access, no production auth.